
UNIT-V

Pointers: Introduction, Pointers and addresses, Pointer types, Pointers and function arguments, Pointers and arrays, address arithmetic, Array of Pointers, Pointers to Pointers, Pointer to Function, pointers and multi-dimensional arrays. Dynamic Memory Allocation.

Strings: String Basics, String Library Functions: Assignment and Substrings, Longer Strings: Concatenation and Whole-Line Input, String Comparison, character pointers and functions

Pointers: Introduction

One of the powerful features of C is ability to access the memory variables by their memory address. This can be done by using Pointers. The real power of C lies in the proper use of Pointers.

A pointer is a variable that can store an address of a variable (i.e., 112300). We say that a pointer points to a variable that is stored at that address. A pointer itself usually occupies 4 bytes of memory (then it can address cells from 0 to 232-1).

Advantages of Pointers:

1. A pointer enables us to access a variable that is defined outside the function.
2. Pointers are more efficient in handling the data tables.
3. Pointers reduce the length and complexity of a program.
4. They increase the execution speed.
5. Passing large data to functions
6. Dynamic memory allocation

Definition:

A variable that holds a physical memory address is called a pointer variable or Pointer.

or

A pointer is a special variable that stores the address of another variable.

Pointer Declaration:

To declare a pointer, use the dereferencing operator (*) followed by the data type.

Syntax:

The general form of a pointer variable declaration is –

type *var-name;

where,

var-name is the name of the pointer.

type is the type of data it is pointing to.

The (*) dereferencing operator is used to denote that the declared variable stores the address of another variable.

Example of Valid Pointer Variable Declarations

```
int *ip; /* pointer to an integer */  
double *dp; /* pointer to a double */  
float *fp; /* pointer to a float */  
char *ch; /* pointer to a character */
```

Pointer Initialization:

After declaring a pointer variable, you need to initialize it with the address of another variable using the address of (&) operator.

This process is known as referencing a pointer.

Syntax:

```
pointer_variable = &variable;
```

Example

```
int x = 10;  
int *ptr = &x;
```

Pointers and Addresses

A pointer references a location in memory. Obtaining the value stored at that location is known as dereferencing the pointer.

The & Operator –

It is also known as the "Address-of operator".

It is used for Referencing which means taking the address of an existing variable (using &) to set a pointer variable.

The * Operator –

It is also known as the "dereference operator".

Dereferencing a pointer is carried out using the * operator to get the value from the memory address that is pointed by the pointer.

In simple

Address Operator (&)

Gives the memory address of a variable.

Dereference Operator (*)

Accesses the value stored at the address.

Examples:

```
#include <stdio.h>

int main() {
    int a = 25;
    int *p = &a;
    printf("Address of a = %p\n", &a);
    printf("Value of p = %p\n", p);
    printf("Value at p = %d\n", *p);
}
```

p and &a are the same

*p gives value of a

Access and Manipulate Values of a Variable through its pointer:

Once a pointer has been assigned address of a variable then access the value of the variable can be done by using *

Ex:

```
int x,*p,n;
x=25;
p=&x;
n=*p;
```

1) first line declares x and n integer variables and p as pointer variable

2) 2nd line declares value assigned to x.

3) 3rd line assigns the address of x to the p.

4) 4th line indicates indirection operator(*) before placed pointer variable then the pointer returns the value of variable.

Example:

```
Int x;
int *p; //declaration of pointer
x =5;
p= &x; //initialization of pointer
```

```
// Assume that x is stored at the memory address 2000
```

```
Printf("The Value of x is %d",x);
```

```
Printf("The Address of x is %u",&x);
```

```
Printf("The Address of x is %u",p);
```

```
Printf("The Value of x is %d",*p);
```

```
Printf("The Value of x is %d",*(&x));
```

Output:

5

2000

2000

5

5

Ex:

```
#include <stdio.h>
```

```
int main() {
```

```
    int x = 10;
```

```
    float y = 1.3f;
```

```
    char z = 'p';
```

```
    // Pointer declaration and initialization
```

```
    int * ptr_x = & x;
```

```
    float * ptr_y = & y;
```

```
    char * ptr_z = & z;
```

```
    // Printing the values
```

```
    printf("Value of x = %d\n", * ptr_x);
```

```
    printf("Value of y = %f\n", * ptr_y);
```

```
    printf("Value of z = %c\n", * ptr_z);
```

```
    return 0;
```

```
}
```

Output:

Value of x = 10

Value of y = 1.300000

Value of z = p

Ex:

```
#include <stdio.h>

int main() {
    int a = 10;
    // declare pointer variable
    int* ptr;
    // note that data type of ptr and var must be same
    ptr = &a;
    // assign the address of a variable to a pointer
    printf("Value at ptr = %p \n", ptr);
    printf("Value at a = %d \n", a);
    printf("Value at *ptr = %d \n", *ptr);
    return 0;
}
```

Output:

Value at ptr = 0x7ffd97e566dc

Value at a = 10

Value at *ptr = 10

Ex:

```
#include <stdio.h>

int main() {
    int x = 10;
    // Pointer declaration and initialization
    int * ptr = &x;
    // Printing the current value
    printf("Value of x = %d\n", * ptr);
    // Changing the value
    * ptr = 20;
```

```
// Printing the updated value
printf("Value of x = %d\n", * ptr);
return 0; }
```

Output

Value of x = 10

Value of x = 20

Size of a Pointer Variable

The memory (or, size) occupied by a pointer variable does not depend on the type of the variable it is pointing to. The size of a pointer depends on the system architecture.

```
#include <stdio.h>

int main() {
    int x = 10;
    float y = 1.3f;
    char z = 'p';

    // Pointer declaration and initialization
    int * ptr_x = & x;
    float * ptr_y = & y;
    char * ptr_z = & z;

    // Printing the size of pointer variables
    printf("Size of integer pointer : %lu\n", sizeof(ptr_x));
    printf("Size of float pointer : %lu\n", sizeof(ptr_y));
    printf("Size of char pointer : %lu\n", sizeof(ptr_z));
    return 0; }
```

Output:

Size of integer pointer : 8

Size of float pointer : 8

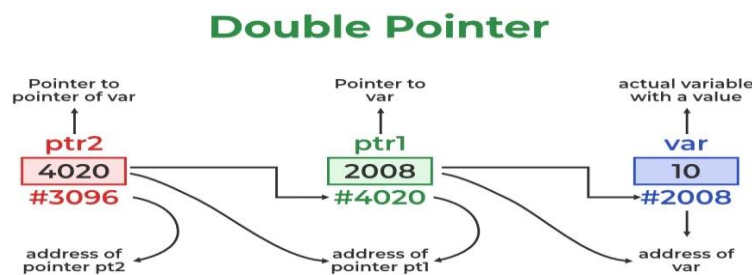
Size of char pointer : 8

POINTERS TO POINTERS (Double Pointer) :

So far, all pointers have been pointing directly to data. It is possible and with advanced data structures often necessary to use pointers to that point to other pointers. A pointer-to-pointer (double pointer) is used to store the address of another pointer.

The first pointer stores the address of a variable, and the second pointer stores the address of the first pointer.

double pointer occupies the same amount of memory as a regular pointer on the stack.



Declaration of Pointer to a Pointer

```
data_type_of_pointer **name_of_variable = & normal_pointer_variable;
```

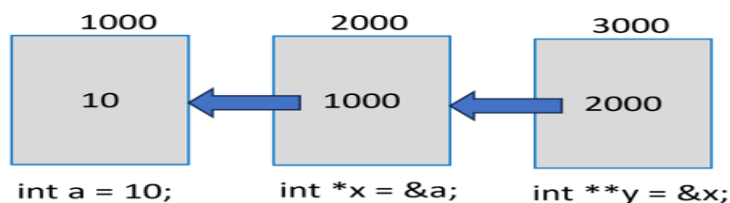
For example, we can have a pointer pointing to a pointer to an integer. This two level indirection is seen as below:

//Local declarations

```
int a;
```

```
int* x;
```

```
int **y;
```



Ex:

```
#include <stdio.h>
```

```
int main() {
```

```
    // An integer variable
```

```
    int a = 100;
```

```
    // Pointer to integer
```

```
    int *ptr = &a;
```

```
    // Pointer to pointer (double pointer)
```

```
    int **dptr = &ptr;
```

```
    printf("Value of 'a' is : %d\n", a);
```

```
    printf("Value of 'a' using pointer (ptr) is : %d\n", *ptr);
```

```
printf("Value of 'a' using double pointer (dptr) is : %d\n", **dptr);  
return 0;  
}
```

Output:

Value of 'a' is : 100

Value of 'a' using pointer (ptr) is : 100

Value of 'a' using double pointer (dptr) is : 100

Ex:

```
#include <stdio.h>  
  
int main(){  
    int var = 10;  
    int *intptr = &var;  
    int **ptrptr = &intptr;  
    printf("var: %d \nAddress of var: %d \n\n", var, &var);  
    printf("inttpr: %d \nAddress of inttpr: %d \n\n", intptr, &intptr);  
    printf("var: %d \nValue at intptr: %d \n\n", var, *intptr);  
    printf("ptrptr: %d \nAddress of ptrptr: %d \n\n", ptrptr, &ptrptr);  
    printf("intptr: %d \nValue at ptrptr: %d \n\n", intptr, *ptrptr);  
    printf("var: %d \n*intptr: %d \n**ptrptr: %d", var, *intptr, **ptrptr);  
    return 0;  
}
```

Output:

var: 10

Address of var: 951734452

inttpr: 951734452

Address of inttpr: 951734456

var: 10

Value at intptr: 10

ptrptr: 951734456

Address of ptrptr: 951734464

intptr: 951734452

Value at ptrptr: 951734452

var: 10

*intptr: 10

**ptrptr: 10

Ex:

```
#include <stdio.h>
```

```
int main() {
```

```
    int var = 789;
```

```
    // pointer for var
```

```
    int* ptr2;
```

```
    // double pointer for ptr2
```

```
    int** ptr1;
```

```
    // storing address of var in ptr2
```

```
    ptr2 = &var;
```

```
    // Storing address of ptr2 in ptr1
```

```
    ptr1 = &ptr2;
```

```
    printf("Value of var = %d\n", var);
```

```
    printf("Value of var using single pointer = %d\n", *ptr2);
```

```
    printf("Value of var using double pointer = %d\n", **ptr1);
```

```
    return 0; }
```

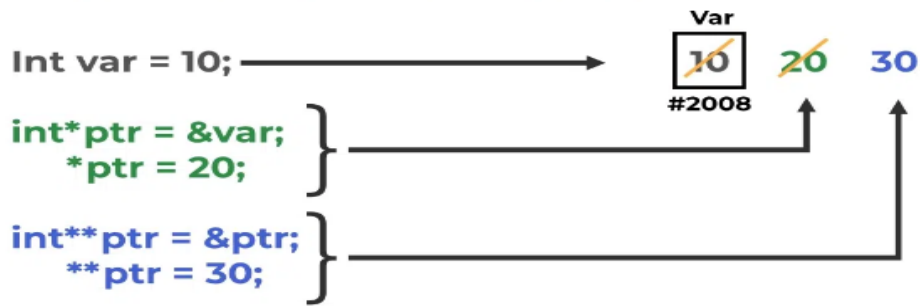
Output:

Value of var = 789

Value of var using single pointer = 789

Value of var using double pointer = 789

Double Pointer Working



Ex:

```
#include <stdio.h>

int main()
{
    // defining single and double pointers
    int a = 5;
    int* ptr = &a;
    int** d_ptr = &ptr;

    // size of single pointer
    printf(" Size of normal Pointer: %d \n", sizeof(ptr));

    // size of double pointer
    printf(" Size of Double Pointer: %d \n", sizeof(d_ptr));

    return 0;
}
```

Output:

Size of normal Pointer: 8

Size of Double Pointer: 8

Pointer Types

Pointer type depends on data type it points to.

```
int *ip;
char *cp;
float *fp;
double *dp;
```

Why pointer type matters?

Because it decides:

How many bytes to read

How address arithmetic works

Example:

```
int a = 10;
int *p = &a;
*p reads 4 bytes (int size).
```

NULL Pointer

The Null Pointer is the pointer that does not point to any location but NULL

The NULL pointer is a constant with a value of "0"

It is always a good practice to assign a NULL value to a pointer variable in case you do not have an exact address to be assigned.

This is done at the time of variable declaration

Syntax of Null Pointer Declaration:

```
type pointer_name = NULL;
```

or

```
type pointer_name = 0;
```

Example:

```
#include <stdio.h>
int main(){
    int *ptr = NULL;
    printf("The value of ptr is : %x\n", ptr);
    return 0;
}
```

Output:

The value of ptr is 0

Check if the pointer is NULL:

It is a valid operation in pointer arithmetic to check whether the pointer is NULL. We just have to use isequal to operator (==) as shown below:

```
ptr == NULL;
```

```
#include <stdio.h>
```

```
int main() {
```

```

// declaring null pointer

int* ptr = NULL;

// derefencing only if the pointer have any value
if (ptr == NULL) {
    printf("Pointer does not point to anything");
}
else {
    printf("Value pointed by pointer: %d", *ptr);
}

return 0;
}

```

OUTPUT

Pointer does not point to anything

pass NULL to a function

```

#include <stdio.h>

void foo(int* string)
{
    if (string == NULL) {
        printf("NULL Pointer Passed");
        return;
    }

    printf("Non-Null Pointer Passed");
}

int main()
{
    foo(NULL);
    return 0;
}

```

OUTPUT:

NULL Pointer Passed

VOID POINTER:

In C, A void pointer or generic pointer is a pointer that has no associated data type with it. A void pointer can hold an address of any type and can be type-casted to any type.

Any pointer to an object may be converted to type void * without loss of information. If the result is converted back to the original type; the original pointer is recovered.

```
#include <stdio.h>

int main()
{
    int a = 10;
    char b = 'x';
    // void pointer holds address of int 'a'
    void* p = &a;
    // void pointer holds address of char 'b'
    p = &b;
}
```

Properties of Void Pointers

1. void pointers cannot be dereferenced.
2. The C standard doesn't allow pointer arithmetic with void pointers. However, in GNU C it is allowed by considering the size of the void as 1.

1. Example:

```
#include <stdio.h>

int main()
{
    int a = 10;
    void* ptr = &a;
    printf("%d", *ptr);
    return 0;
}
```

OUTPUT

Compiler Error: 'void*' is not a pointer-to-object type

// C program to dereference the void pointer to access the value

```

#include <stdio.h>

int main()
{
    int a = 10;
    void* ptr = &a;
    // The void pointer 'ptr' is cast to an integer pointer
    // using '(int*)ptr' Then, the value is dereferenced
    // with `*(int*)ptr` to get the value at that memory location
    printf("%d", *(int*)ptr);
    return 0;
}

```

Output:

10

2. Example:

// C program to demonstrate the usage of a void pointer to perform pointer arithmetic and access a specific memory location

```

#include <stdio.h>

int main() {
    int a[2] = { 1, 2 };
    // Declare a void pointer and assign the address of array 'a' to it
    void* ptr = &a;
    // Increment the pointer by the size of an integer
    ptr = ptr + sizeof(int);
    // The void pointer 'ptr' is cast to an integer pointer using '(int*)ptr' Then, the value is //
    dereferenced with `*(int*)ptr` to get the value at that memory location
    printf("%d", *(int*)ptr);
    return 0; }

```

Output:

2

Difference Between NULL Pointer and Void Pointer

NULL Pointer	Void Pointer
A NULL pointer does not point to anything. It is a special reserved value for pointers.	A void pointer points to the memory location that may contain type-less data.
Any pointer type can be assigned NULL.	It can only be of type void.
All the NULL pointers are equal.	Void pointers can be different.
NULL Pointer is a value.	A void pointer is a type.
Example: <code>int *ptr = NULL;</code>	Example: <code>void *ptr;</code>

Pointers and Function Arguments:

Parameter passing mechanism in C is of two types.

- 1.Call by Value
- 2.Call by Reference.

The process of passing the actual value of variables is known as Call by Value. The process of calling a function using pointers to pass the addresses of variables is known as Call by Reference. The function which is called by reference can change the value of the variable used in the call.

Example of Call by Value (default) Original value unchanged:

```
#include <stdio.h>

void swap(int,int);

main()
{
    int a,b;
    printf("Enter the Values of a and b:");
    scanf("%d%d",&a,&b);
    printf("Before Swapping \n");
    printf("a = %d \t b = %d", a,b);
    swap(a,b);
    printf("After Swapping \n");
```

```

    printf("a = %d \t b = %d", a,b);
}

void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}

```

OUTPUT:

Enter the Values of a and b: 10 20

Before Swapping

a = 10 b = 20

After Swapping

a = 10 b = 20

Example of Call by Reference:

```

#include<stdio.h>

main()
{
    int a,b;
    a = 10;
    b = 20;
    swap (&a, &b);
    printf("After Swapping \n");
    printf("a=%d \t b =%d",a,b);
}

void swap(int *x, int *y)
{
    int temp;
    temp = *x;

```



```
*x = *y;
*y = temp;
}
```

output:

After Swapping

a=20 b =10

Pointers and Arrays

An array name acts like a pointer constant.

The value of this pointer constant is the address of the first element.

For example, if we have an array named arr then arr and &arr[0] can be used interchangeably.

```
#include <stdio.h>
```

```
int main() {
    // Declare an array
    int arr[3] = { 5, 10, 15 };
    // Access first element using index
    printf("%d\n", arr[0]);
    // Access first element using pointer
    printf("%d\n", *arr);
    return 0;
}
```

Output:

5

5

```
int arr[3] = {10, 20, 30};
```

```
int *p = arr;
```

<u>Expression</u>	<u>Meaning</u>
-------------------	----------------

arr	address of arr[0]
-----	-------------------

p+1	next element
-----	--------------

*(p+1)	value of arr[1]
--------	-----------------

```
printf("%d", *(p+2)); // 30
```

Traversing Array Using Pointer to First Element

```
#include <stdio.h>

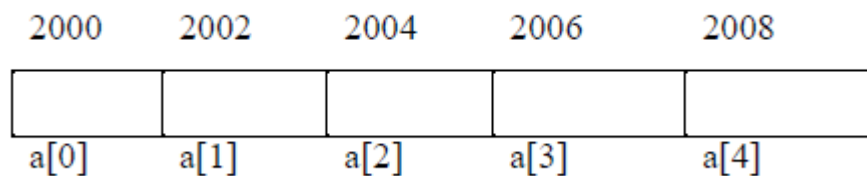
int main() {
    int arr[5] = { 1, 2, 3, 4, 5 };
    int n = sizeof(arr) / sizeof(arr[0]);
    // Defining the pointer to first element of array
    int* ptr = &arr[0];
    // Traversing array using pointer arithmetic
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);
    return 0;
}
```

Output:

1 2 3 4 5

Passing Array

When an array is declared, elements of array are stored in contiguous locations. The address of the first element of an array is called its base address.



The name of the array is called its base address.

both a and a[0] points to location 2000. If we declare P as an integer pointer, then we can make the pointer P to point to the array a by following assignment

P = a;

We can access every value of array a by moving P from one element to another.

i.e.,

P points to 0th element

P+1 points to 1st element

P+2 points to 2nd element

P+3 points to 3rd element

P +4 points to 4th element

Syntax:

`*(arr + i)`

`*(arr + i)` is the same as `arr[i]` in a C Program.

`*` is a dereferencing operator used to extract the value from the address `(arr + i)`.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[5] = {2, 4, 6, 8, 10}, i;
```

```
    for(i = 0; i < 5; i++)
```

```
    {
```

```
        // printing the elements address and value at arr[i] using *(arr + i)
```

```
        printf("[index %d] Address : %u, Value : %d\n", i, (arr + i), *(arr + i));
```

```
    }
```

```
    return 0;
```

```
}
```

OUTPUT:

```
[index 0] Address : 2364420656, Value : 2
```

```
[index 1] Address : 2364420660, Value : 4
```

```
[index 2] Address : 2364420664, Value : 6
```

```
[index 3] Address : 2364420668, Value : 8
```

```
[index 4] Address : 2364420672, Value : 10
```

Array name is a pointer

```
int arr[5] = {10, 20, 30, 40, 50};
```

```
arr == &arr[0]
```

```
arr + 1 == &arr[1]
```

Accessing array using pointers

```
printf("%d", *(arr + 2)); // 30
```

Using pointer variable

```
int *p = arr;
```

```
printf("%d", p[3]); // 40
```

✓ $p[i] \equiv *(p + i)$

Pointers and 2D Array (Multi-Dimensional Arrays)

In a two-dimensional array, we can access each element by using two subscripts.

Suppose arr is a 2-D array, we can access any element $arr[i][j]$ of the array using the pointer expression:

$*(*(arr + i) + j)$.

Let's take a two-dimensional array $arr[3][4]$:

```
int arr[3][4] = { { 1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} };
```

Dereferencing this expression provides the j th element of the i th row. Hence, we get:

$*(*(arr + i) + j)$.

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[3][4] = { { 1, 2, 3, 4}, {5, 6, 7, 8},
```

```
                    {9, 10, 11, 12} }; // pointer to an array of 4 integers
```

```
    for (int i = 0; i < 3; i++) {
```

```
        for (int j = 0; j < 4; j++)
```

```
            // using pointer notation for 2D array
```

```
            printf("%d ", *(*(arr + i) + j));
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

1 2 3 4

5 6 7 8

9 10 11 12

Expression	What it gives
arr	pointer to row 0

<code>arr + i</code>	pointer to row i
<code>*(arr + i)</code>	row i (array $\rightarrow$ decays to pointer)
<code>*(arr + i) + j</code>	address of element <code>[i][j]</code>
<code>*(*(arr + i) + j)</code>	value at <code>[i][j]</code>

Address Arithmetic (Pointer Arithmetic)

Pointer arithmetic means performing arithmetic operations on pointers to move across memory locations based on the size of the data type the pointer points to.

Allowed operations on pointers

1. Increment and Decrement of a Pointer
2. Addition and Subtraction of Integer to Pointer
3. Subtraction of Pointers
4. Comparison of Pointers

Note:

For `int *p`:

Operation	Moves
<code>p+1</code>	4 bytes (int)
<code>p+2</code>	8 bytes

Pointer arithmetic depends on data type

Only these are valid in C:

Operation	Example
Addition	<code>p + 1</code>
Subtraction	<code>p - 1</code>
Increment	<code>p++</code>
Decrement	<code>p--</code>
Difference of two pointers	<code>p2 - p1</code> (same array only)
Comparison	<code>p1 < p2</code>

Why pointer arithmetic depends on data type:

Example

```
int *p;
```

If: $p = 1000$

$\text{sizeof}(\text{int}) = 4$

Then:

$p + 1 = 1004$

$p + 2 = 1008$

Compiler internally does:

$\text{new_address} = \text{old_address} + (n \times \text{sizeof}(\text{type}))$

Increment and Decrement of a Pointer

```
#include <stdio.h>
```

```
int main() {  
    int arr[] = {10, 20, 30};  
    int *p = arr;  
    printf("p points to %d\n", *p);  
    p++; // move to next integer  
    printf("After p++: %d\n", *p);  
    p--; // move back  
    printf("After p--: %d\n", *p);  
    return 0;  
}
```

Output:

10

20

10

Addition and Subtraction of Integer to Pointer

```
#include <stdio.h>
```

```
int main() {  
    int arr[] = {5, 15, 25, 35};  
    int *p = arr;
```

```
printf("%d\n", *(p + 2)); // access 3rd element
printf("%d\n", *(p - 0)); // same as *p
return 0;
}
```

Output:

25

5

Ex:

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int *p = &arr[3]; // points to 40
    printf("%d\n", *(p - 1)); // arr[2]
    printf("%d\n", p[-2]);    // arr[1]
    return 0;
}
```

Output:

30

20

Subtraction of Pointers

```
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int *p1 = &arr[1];
    int *p2 = &arr[4];
    printf("%ld\n", p2 - p1);
    return 0;
}
```

Explanation:

Both pointers must point to same array

Result = number of elements between them

Output

3

Comparison of Pointers

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40};
    int *p1 = &arr[1];
    int *p2 = &arr[3];
    if (p1 < p2)
        printf("p1 is before p2\n");
    if (p2 > p1)
        printf("p2 is after p1\n");
    return 0;
}
```

Explanation:

Pointer comparison is valid only within same array

Compares addresses, not values

Output

p1 is before p2

p2 is after p1

INVALID Pointer Arithmetic

p + p; // ✗ invalid

p * 2; // ✗ invalid

p / 2; // ✗ invalid

void *vp;

vp++; // ✗ invalid in standard C

Array of Pointers

Stores addresses, not actual data.

```
char *names[3] = {"Ram", "Sam", "Tom"};
```

Memory efficient compared to 2D array

```
char names[3][10]; // wastes space
```

Access

```
printf("%s", names[1]); // Sam
```

```
printf("%c", names[1][0]); // S // (*(names + 1) + 0)
```

Pointer to Function or Function Pointer

A function pointer is a pointer that stores the address of a function.

Just like:

int *p → points to an int

void *p → points to any data

int (*fp)(int, int) → points to a function

General syntax

```
return_type (*pointer_name)(parameter_types);
```

Used in:

- Callbacks
- Menu-driven programs

Ex:

```
#include <stdio.h>
```

```
int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

```
int main() {
```

```
    int (*fp)(int, int); // function pointer declaration
```

```
    fp = add;           // or fp = &add;
```

```
    printf("%d\n", fp(2, 3)); // function call via pointer //5
```

```
    printf("%d\n", (*fp)(4, 5)); //9
```

```
    return 0; }
```

Passing Function Pointer as Argument (Callback)

```
#include <stdio.h>

int multiply(int x, int y) {
    return x * y;
}

int compute(int a, int b, int (*operation)(int, int)) {
    return operation(a, b); //operation is a function pointer parameter
}

int main() {
    printf("%d\n", compute(3, 4, multiply)); //12
    return 0;
}
```

Array of Function Pointers (Menu-Driven)

```
#include <stdio.h>

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }

int main() {
    int (*ops[3])(int, int) = { add, sub, mul };
    printf("%d\n", ops[0](10, 5)); // add  15
    printf("%d\n", ops[1](10, 5)); // sub   5
    printf("%d\n", ops[2](10, 5)); // mul  50
    return 0;
}
```

Passing array of Strings to function

Array of Function Pointers

```
#include <stdio.h>

// Function receives pointer to an
//array of 50 characters
void display(char (*p)[50], int n)
```

```

{
    for (int i = 0; i < n; i++)
    {
        printf("%s\n", p[i]);
    }
}

int main()
{
    char names[3][50] ;
    for(int i=0;i<3;i++)
    {
        printf("\n enter %d name\n ",i);
        scanf("%s",names[i]);
    }

    //Pointer to the string array
    char (*p)[50];
    // Assign array to pointer
    p = names;
    // Pass pointer to function
    display(p, 3);
    return 0;
}

int *fp(int, int); // ✗ declares a function returning int*
int (*fp)(int, int); // ✔ function pointer

```

comparison

Declaration	Meaning
char (*p)[50];	pointer to array of 50 chars
char *p[50];	array of 50 char pointers
char (*fp)(int);	function pointer

char *fp(int);	function returning char pointer
----------------	---------------------------------

DYNAMIC MEMORY ALLOCATION:

Dynamic memory allocation uses predefined functions to allocate and release memory for data while the program is running. It effectively postpones the data definition, but not the declaration to run time.

To use dynamic memory allocation, we use either standard data types or derived types. To access data in dynamic memory we need pointers.

Understanding Memory Management

two types of memory:

static memory and dynamic memory.

When allocated memory blocks during compile-time, it is stored in static memory and referred to as static memory allocation.

Conversely, memory allocated during runtime is stored in dynamic memory and is known as dynamic memory allocation.

Static Memory Allocation	Dynamic Memory Allocation
Allocated at compile-time	Allocated at runtime
Memory size is fixed and predetermined	Memory size can be changed during runtime
Memory is stored in the stack section of memory	Memory is stored in the heap section of memory
Memory is automatically deallocated when a function returns or the program exits	Memory must be explicitly deallocated using functions such as free()
Cannot handle unpredictable data size	Can handle unpredictable data size by allocating memory as needed

Dynamic Memory Allocation refers to the process of allocating or deallocating memory blocks during a program's runtime.

This is achieved through the use of **four functions**:

malloc()

calloc()

free()

realloc()

These 4 library functions defined under `<stdlib.h>` header file to facilitate dynamic memory allocation

malloc() method

The “malloc” or “memory allocation” method in C is used to dynamically allocate a single large block of memory with the specified size.

It returns a pointer of type void which can be cast into a pointer of any form.

Syntax

```
void *malloc(size_t size);
```

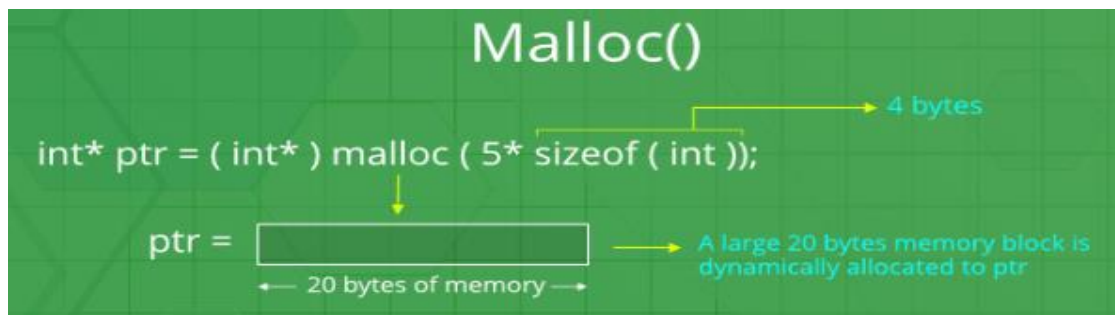
or

```
ptr = (cast-type*) malloc(byte-size)
```

For Example:

```
ptr = (int*) malloc(100 * sizeof(int));
```

malloc() returns a pointer to the first byte of the allocated memory block. If the memory allocation fails, it returns a NULL pointer.



```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int* ptr;    int n, i;
```

```
    printf("Enter number of elements:");
```

```
    scanf("%d",&n);
```

```
    printf("Entered number of elements: %d\n", n);
```

```
    ptr = (int*)malloc(n * sizeof(int));
```

```
    if (ptr == NULL) {
```

```
        printf("Memory not allocated.\n");
```

```
        exit(0);    }
```

```
    else {
```

```
        printf("Memory successfully allocated using malloc.\n");
```

```
        for (i = 0; i < n; ++i) {
```

```

    ptr[i] = i + 1; }

printf("The elements of the array are: ");

for (i = 0; i < n; ++i) {
    printf("%d, ", ptr[i]);    } }

return 0; }

```

Output:

Enter number of elements:7

Entered number of elements: 7

Memory successfully allocated using malloc.

The elements of the array are: 1, 2, 3, 4, 5, 6, 7

calloc() method

“calloc” or “contiguous allocation” method is used to dynamically allocate the specified number of blocks of memory of the specified type.

it is very much similar to malloc() but has two different points and these are:

It initializes each block with a default value ‘0’.

It has two parameters or arguments as compare to malloc().

Syntax:

```
ptr = (cast-type*)calloc(n, element-size);
```

here, n is the no. of elements

element-size is the size of each element.

For Example:

```
ptr = (float*) calloc(25, sizeof(float));
```

This statement allocates contiguous space in memory for 25 elements each with the size of the float.



```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```

int* ptr;
int n, i;
printf("Enter number of elements:");
scanf("%d",&n);
ptr = (int*)calloc(n, sizeof(int));
if (ptr == NULL) {
    printf("Memory not allocated.\n");
    exit(0); }
else {
    printf("Memory successfully allocated using calloc.\n");
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;    }
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);    } }
return 0; }

```

Output:

```

Enter number of elements: 5
Memory successfully allocated using calloc.
The elements of the array are: 1, 2, 3, 4, 5

```

free() method

“free” method is used to dynamically de-allocate the memory.

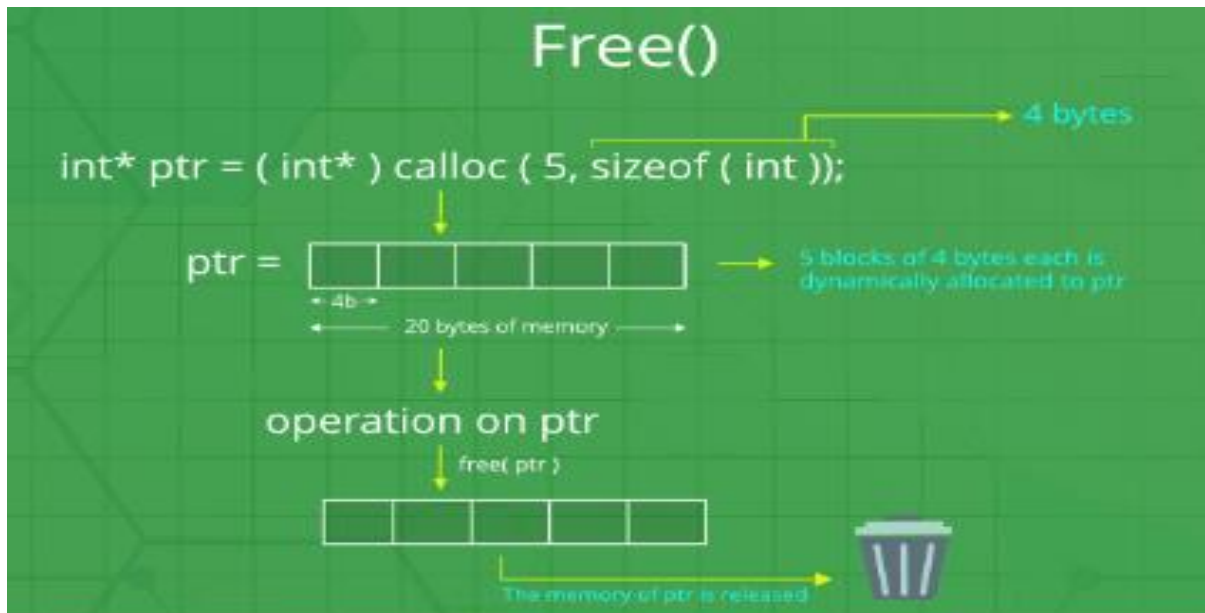
The memory allocated using functions malloc() and calloc() is not de-allocated on their own.

Hence the free() method is used, whenever the dynamic memory allocation takes place.

It helps to reduce wastage of memory by freeing it.

Syntax:

```
free(ptr);
```



```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int *ptr, *ptr1;
    int n, i;
    printf("Enter number of elements:");
    scanf("%d",&n);
    ptr = (int*)malloc(n * sizeof(int));
    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);    }
    else {
        printf("Memory successfully allocated using malloc.\n");
        free(ptr);
        printf("Malloc Memory successfully freed.\n");
    }
    return 0;
}
```

Output:

Enter number of elements: 5

Memory successfully allocated using malloc.

Malloc Memory successfully freed.

realloc() method

“realloc” or “re-allocation” method is used to dynamically change the memory allocation of a previously allocated memory.

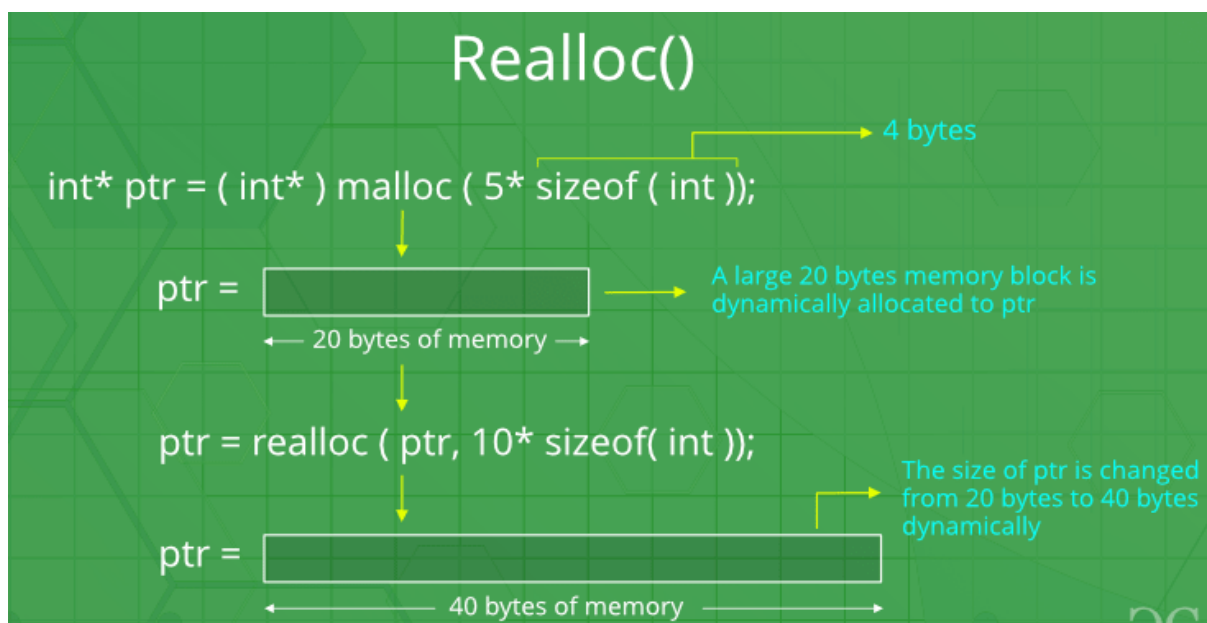
In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to dynamically re-allocate memory.

re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

Syntax:

```
ptr = realloc(ptr, newSize);
```

where ptr is reallocated with new size 'newSize'.



```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int* ptr;
```

```
    int n, i;
```

```
    n = 5;
```

```
    printf("Enter number of elements: %d\n", n);
```

```
    ptr = (int*)calloc(n, sizeof(int));
```

```

if (ptr == NULL) {
    printf("Memory not allocated.\n");
    exit(0); }
else {
    printf("Memory successfully allocated using calloc.\n");
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;    }
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);    }
    n = 10;
    printf("\n\nEnter the new size of the array: %d\n", n);
    ptr = (int*)realloc(ptr, n * sizeof(int));
    if (ptr == NULL) {
        printf("Reallocation Failed\n");
        exit(0);    }
    printf("Memory successfully re-allocated using realloc.\n");
    for (i = 5; i < n; ++i) {
        ptr[i] = i + 1;
    }
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);    }
    free(ptr);    }
return 0; }

```

Output:

Enter number of elements: 5

Memory successfully allocated using calloc.

The elements of the array are: 1, 2, 3, 4, 5,

Enter the new size of the array: 10

Memory successfully re-allocated using realloc.

The elements of the array are: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10